

Modulo de Ayuda

Guia didactica del flujo, conceptos base, patron Facade y SRP

Proposito	Explicar como se abre la seccion de ayuda, que ocurre al cargarla, que pasa al tocar Preguntar y por que el diseno separa responsabilidades de manera correcta.
------------------	---

Campo	Detalle
Proyecto	Proyecto movilidad
Modulo	Ayuda
Archivos clave	vistas/ayuda.py, Controladores/controlador_ayuda.py, Servicios/Ayuda/servicio_ayuda.py, contenido_ayuda.py, cliente_gemini.py
Fecha	25 de junio de 2026

Indice

- 1. Mapa general
- 2. Conceptos que conviene saber
- 3. Flujo desde navegacion
- 4. Carga inicial y paneles
- 5. Flujo al tocar Preguntar
- 6. Controlador, Facade y SRP
- 7. ContenidoAyuda y ClienteGemini
- 8. Checklist de estudio

1. Mapa general del modulo

Idea central

Ayuda tiene dos caras: una guia fija por rol y un asistente IA. La vista no decide de donde salen los datos: solo pide una solicitud al controlador y muestra el resultado.

Capa	Clase / archivo	Responsabilidad
Navegacion	RutaAyuda	Abre la pantalla, obtiene usuario y define destino de Volver.
Vista	VistaAyuda y paneles	Construye la interfaz y muestra guia, chat y estados.
Acciones	AccionesBotonesAyuda	Responde a Volver y Preguntar.
Controlador	ControladorAyuda	Unica puerta: pedir_solicitud(). Devuelve ResultadoOperacion.
Servicio	ServicioAyuda	Facade: contenido fijo o Gemini segun la solicitud.
Contenido	ContenidoAyuda	Textos, sugerencias y rol por tipo de usuario.
Cliente	ClienteGemini	HTTP, API key, reintentos y errores.

```
Boton Ayuda -> navegar('ayuda') -> RutaAyuda.ejecutar() -> VistaAyuda
VistaAyuda -> pedir_solicitud(None, usuario) -> contenido inicial
Preguntar -> pedir_solicitud(texto, usuario) -> ServicioAyuda -> ClienteGemini
```

2. Conceptos que conviene saber

Concepto	Que significa en este modulo
Funcion / metodo	Bloque con nombre que ejecuta una tarea. En una clase se llama metodo, por ejemplo <code>tocar_boton_preguntar()</code> .
Clase	Molde de objetos. <code>VistaAyuda</code> , <code>ServicioAyuda</code> y <code>ClienteGemini</code> agrupan datos y comportamiento.
Atributo	Dato guardado en un objeto: <code>self.usuario_actual</code> , <code>self.controlador</code> , <code>self.panel_derecho</code> .
Callback	Funcion que se ejecuta despues de un evento. Un boton llama a una accion cuando el usuario lo presiona.
lambda	Funcion pequena anonima. Se usa para pasar la sugerencia correcta a <code>usar_sugerencia(texto)</code> .
try / except	Captura errores. El controlador evita que una excepcion rompa la vista y devuelve <code>ResultadoOperacion</code> .
with	Context manager. En <code>ClienteGemini</code> abre la respuesta HTTP y la cierra al salir del bloque.
threading.Thread	Ejecuta la consulta a Gemini en segundo plano para no congelar la ventana.
after	Metodo de Tkinter para volver al hilo de la interfaz y actualizar widgets de forma segura.
dataclass	Forma corta de crear clases de datos. <code>PerfilAyuda</code> guarda secciones y sugerencias por rol.

3. Flujo desde navegacion hasta abrir Ayuda

La ayuda se abre desde botones que llaman a navegar('ayuda'). Si no hay sesion, Volver apunta a pantalla_inicial; si hay usuario, Volver apunta al menu.

N	Que ocurre	Archivo / clase
1	El usuario toca Ayuda; el comando llama a navegar('ayuda').	pantalla_inicial.py o menu.py
2	Navegacion.navegar delega en NavegadorRutas.	Controladores/navegacion.py
3	NavegadorRutas busca la ruta registrada con destino ayuda.	NavegadorRutas
4	RutaAyuda.ejecutar limpia pantalla, cambia titulo y obtiene usuario actual.	RutaAyuda
5	Se instancia VistaAyuda con ventana, navegar, controlador, usuario y destino_volver.	VistaAyuda

```
usuario_actual = obtener_usuario_actual()
destino_volver = 'menu' if usuario_actual is not None else 'pantalla_inicial'
```

4. Carga inicial y paneles

Al construirse, VistaAyuda pide el contenido inicial usando el mismo metodo del controlador. La solicitud None significa: traer rol, secciones y sugerencias; no consultar Gemini.

Elemento	Funcion
VistaAyuda.__init__	Guarda dependencias y llama a controlador.pedir_solicitud(None, usuario).
ServicioAyuda	Detecta None y devuelve contenido fijo desde ContenidoAyuda.
PanelIzquierdoAyuda	Muestra guia para el rol y lista secciones.
PanelDerechoAyuda	Crea encabezado IA, bienvenida, sugerencias, entrada y chat.

Detalle La carga inicial funciona aunque Gemini falle, porque no hace request externo: solo consulta ContenidoAyuda.

5. Flujo al tocar Preguntar

El boton no llama directo al servicio. Primero actualiza la pantalla, luego consulta por controlador en segundo plano, y finalmente vuelve al hilo de Tkinter con after.

N	Dentro de tocar_boton_preguntar
1	Obtiene panel_derecho y lee panel.obtener_pregunta().
2	Si no hay texto, muestra 'Escribe una consulta' y termina.
3	Inicia el chat, agrega el mensaje del usuario, limpia entrada y deshabilita el envio.
4	Crea una funcion local pedir_solicitud para ejecutarla en threading.Thread.
5	El hilo llama a controlador.pedir_solicitud(pregunta, usuario_actual).
6	Usa self.vista.after(0, mostrar_resultado) para volver a la UI.
7	Muestra 'Asistente' si fue exitoso o 'Sistema' si hubo error, y reactiva el boton.

```
threading.Thread(target=pedir_solicitud, daemon=True).start()  
resultado = controlador.pedir_solicitud(pregunta, usuario_actual)  
self.vista.after(0, mostrar_resultado)
```

Por que Thread + after Thread evita congelar la ventana mientras Gemini responde. after evita actualizar widgets desde un hilo ajeno a Tkinter.

6. Controlador y patron Facade

ControladorAyuda quedo con un solo metodo publico: pedir_solicitud. Su tarea no es decidir contenido ni llamar Gemini directamente; solo adapta el resultado del servicio a ResultadoOperacion.

```
def pedir_solicitud(self, solicitud=None, usuario=None):
    try:
        respuesta = self.servicio_ayuda.pedir_solicitud(solicitud, usuario)
        return ResultadoOperacion(datos=respuesta)
    except Exception as error:
        return ResultadoOperacion(error=str(error))
```

Facade

Un Facade ofrece una puerta simple hacia un subsistema mas complejo. ServicioAyuda recibe una solicitud unica y decide internamente si usa ContenidoAyuda, validacion o ClienteGemini.

Solicitud	Decision del servicio	Colaborador
None	Crear contenido inicial para pintar la vista.	ContenidoAyuda
Texto vacio	Lanzar ValueError con mensaje claro.	Validacion interna
Pregunta	Crear instruccion contextual y consultar IA.	ContenidoAyuda + ClienteGemini

Por que esta bien aca

- Reduce acoplamiento: la vista no sabe si hay Gemini, perfiles, prompts o errores HTTP.
- Simplifica el controlador: solo tiene pedir_solicitud.
- Centraliza la regla: None significa contenido inicial; texto significa pregunta.
- Permite cambiar Gemini por otro proveedor sin modificar la vista.

7. ContenidoAyuda y ClienteGemini

ContenidoAyuda mantiene informacion fija por rol. ClienteGemini se encarga de la comunicacion externa. Separarlos evita mezclar textos de la aplicacion con detalles HTTP.

Pieza	Responsabilidad
PerfilAyuda	dataclass que guarda secciones, sugerencias e incluir_comunes.
listar(usuario)	Devuelve secciones completas para el rol.
listar_sugerencias(usuario)	Devuelve preguntas rapidas por rol.
como_contexto(usuario)	Convierte secciones a texto para el prompt de Gemini.
ClienteGemini.generar_respuesta	Arma JSON, envia POST, maneja reintentos y extrae texto.

Conceptos concretos en ClienteGemini

Concepto	Donde aparece	Para que sirve
with	urlopen(...) as respuesta	Cierra el recurso HTTP de forma segura.
HTTPError	except urllib.error.HTTPError	Captura 401, 403, 429 y 503.
URLError	except urllib.error.URLError	Captura problemas de conexion.
JSON	json.dumps / json.loads	Convierte diccionarios a texto JSON y viceversa.
401	_mensaje_error_http	Muestra que la clave Gemini no es valida.

Patron cercano

ContenidoAyuda usa una Strategy liviana guiada por datos: cada rol tiene un PerfilAyuda. No hacen falta muchas clases si los perfiles son datos estables.

8. SRP aplicado al modulo

SRP, Single Responsibility Principle, dice que una clase debe tener una razon principal para cambiar. El modulo Ayuda lo aplica separando interfaz, acciones, coordinacion, contenido y API externa.

Unidad	Responsabilidad	Razon para cambiar
VistaAyuda	Ensamblar la pantalla.	Cambio de layout general.
PanelIzquierdoAyuda	Construir guia fija.	Cambio visual del panel de secciones.
PanelDerechoAyuda	Construir asistente visual.	Cambio visual del chat, entrada o sugerencias.
AccionesBotonesAyuda	Responder a botones.	Cambio del comportamiento al tocar Volver o Preguntar.
ControladorAyuda	Adaptar a ResultadoOperacion.	Cambio del contrato vista-controlador.
ServicioAyuda	Coordinar contenido e IA como Facade.	Cambio de flujo o proveedor.
ContenidoAyuda	Mantener contenido por rol.	Cambio de textos o perfiles.
ClienteGemini	Hablar con Gemini.	Cambio de API, claves, errores o modelo.

Por que importa

Si manana cambias Gemini por otro cliente, deberias tocar ClienteGemini y quizas DependenciasAplicacion. La vista no deberia enterarse.

Checklist de estudio

- Desde que pantallas se abre Ayuda y cual es el destino de Volver?
- Por que pedir_solicitud(None, usuario) no consulta Gemini?
- Que problema evita threading.Thread?
- Por que se usa after para mostrar la respuesta?
- Por que ServicioAyuda calza como Facade?
- Que responsabilidad tiene cada clase segun SRP?

Resumen: Ayuda abre una vista con contenido inicial por rol y un asistente IA; la vista dispara acciones, el controlador ofrece una unica entrada, el servicio funciona como Facade, ContenidoAyuda aporta contexto y ClienteGemini resuelve la comunicacion externa.